ARCHITECTURES

Presented By:
DR. Ahmed Al-Badawy
Lecturer, Computer Science Department

LECTURE TOPICS

1. Foundations of Distributed Systems
2. Transparency and Its Dimensions
3. Scalability and Growth Mechanisms
4. Architectural and System Challenges
5. Types of Distributed Systems
6. Industrial Case Studies
7. Summary

FOUNDATIONS OF DISTRIBUTED SYSTEMS 1. INTRODUCTION TO DISTRIBUTED SYSTEMS

Distributed systems enable multiple autonomous computers to collaborate over a network while appearing as a single unified system. They form the backbone of modern cloud platforms, global web services, financial systems, and IoT infrastructures.

- Multiple independent nodes
- Network-based coordination
- Single-system user view
- Foundation of modern infrastructure

2. EVOLUTION OF COMPUTING MODELS

Computing evolved from centralized mainframes to client-server models and finally to fully distributed architectures. Each step increased scalability and flexibility while introducing additional coordination complexity.

- Mainframe era
- Client-server transition
- Cluster and grid computing
- Cloud and edge computing

3. FORMAL DEFINITION

A distributed system is a collection of independent computers that appears to users as a single coherent system. This definition highlights autonomy and transparency as the two defining properties.

- Autonomous components
- Unified logical interface
- Message-based communication
- No shared memory

4. KEY CHARACTERISTICS

Distributed systems differ fundamentally from centralized systems because they lack global state and synchronized clocks. They must handle concurrency and partial failures inherently.

- Concurrency
- Partial failures
- No global clock
- Network latency variability

5. WHY DISTRIBUTED SYSTEMS?

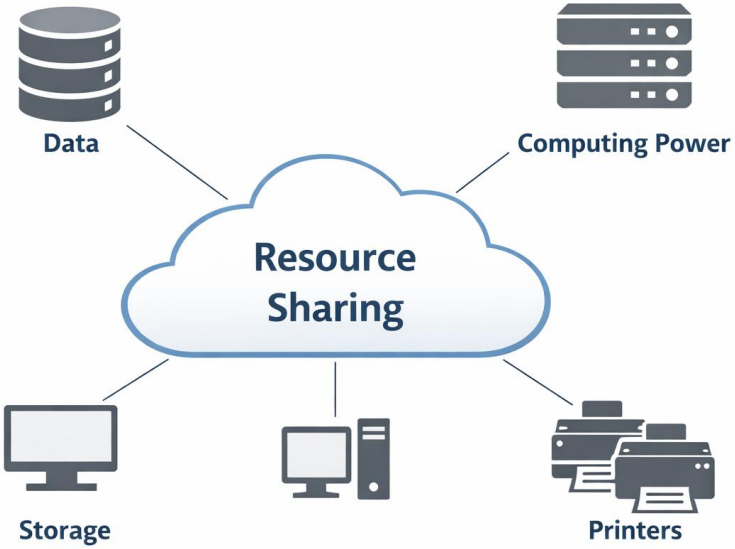
Organizations adopt distributed systems to improve availability, scale to millions of users, and reduce infrastructure costs using commodity hardware instead of centralized supercomputers.

- High availability
- Geographic reach
- Cost efficiency
- Elastic scalability

6. RESOURCE SHARING

Resource sharing allows remote access to storage, databases, processing power, and applications as if they were local. This requires naming, access control, and coordination mechanisms.

- Shared data
- Shared services
- Access control
- Remote invocation



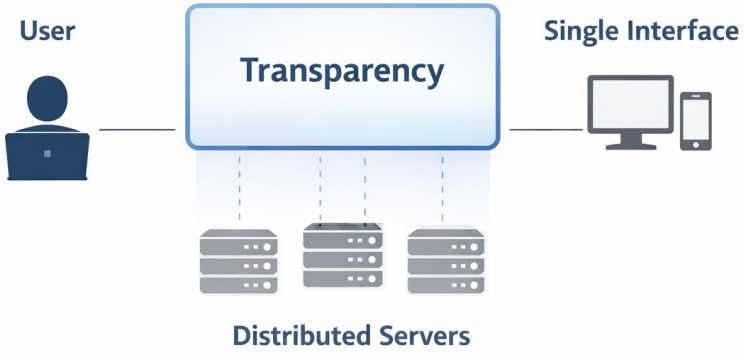
TRANSPARENCY AND ITS DIMENSIONS 7. TRANSPARENCY OVERVIEW

Transparency hides the internal distribution of components, simplifying user interaction while preserving architectural flexibility.

- User abstraction

- Simplified programming

- Reduced complexity exposure



8. ACCESS TRANSPARENCY

Access transparency hides differences in data representation and access methods between local and remote resources.

- Uniform operations
- Location independence
- Interface abstraction

9. LOCATION TRANSPARENCY

Users do not need to know where resources are physically located within the distributed environment.

- Global addressing
- Dynamic resource movement
- Improved scalability

10. REPLICATION TRANSPARENCY

Multiple copies of data or services may exist for performance and reliability without users being aware of them.

- Improved availability
- Load balancing
- Consistency challenges

11. MIGRATION TRANSPARENCY

Resources can move between nodes without affecting ongoing operations or user access patterns.

- Dynamic scaling
- Load redistribution
- Service continuity

12. FAILURE TRANSPARENCY

Systems attempt to mask component failures to maintain uninterrupted service availability.

- Redundancy
- Automatic recovery
- Monitoring mechanisms

13. LIMITS OF TRANSPARENCY

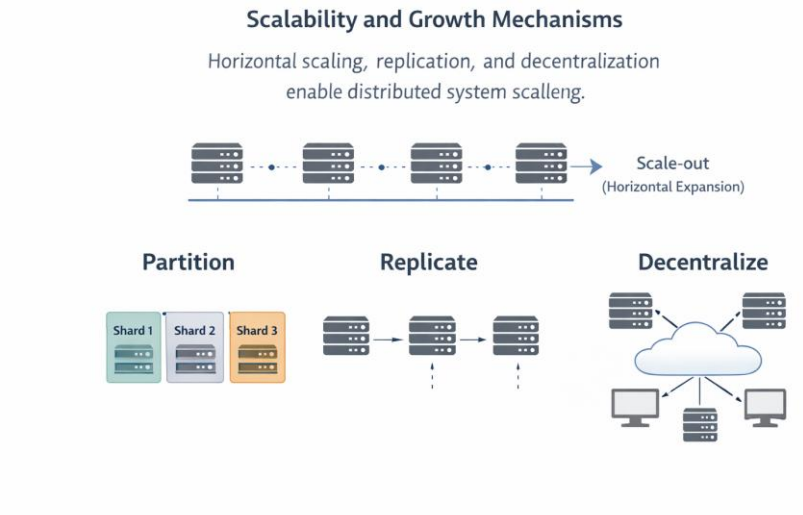
Complete transparency may reduce performance and increase complexity. In some cases, exposing distribution improves clarity.

- Performance trade-offs
- Debugging complexity
- Architectural overhead

SCALABILITY AND GROWTH MECHANISMS
14. OPENNESS IN DISTRIBUTED SYSTEMS

Open distributed systems rely on standardized interfaces and protocols to ensure interoperability and long-term evolution.

- Standard protocols
- Interoperability
- Extensibility



15. SCALABILITY CONCEPT

Scalability ensures that a system can grow in size and geographic distribution without major redesign.

- Horizontal scaling
- Elastic expansion
- Performance stability

16. SIZE SCALABILITY

Systems must handle increasing numbers of users, services, and data volumes efficiently.

- User growth
- Data growth
- Service expansion

17. GEOGRAPHIC SCALABILITY

Distributed systems operate across multiple regions and continents while maintaining acceptable latency.

- Wide-area networks
- Data center distribution
- Latency management

18. ADMINISTRATIVE SCALABILITY

Multiple organizations or domains may operate within the same distributed system.

- Federated systems
- Cross-domain trust
- Governance complexity

19. REPLICATION AS A SCALABILITY TOOL

Replication improves reliability and read performance but introduces synchronization complexity.

- High availability
- Fault tolerance
- Consistency trade-offs

20. PARTITIONING (SHARDING)

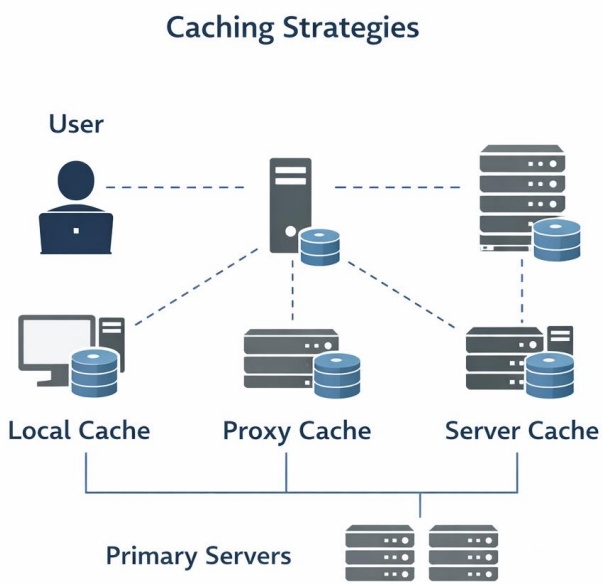
Partitioning divides data or services across nodes to distribute load and improve performance.

- Load distribution
- Parallel processing
- Reduced bottlenecks

21. CACHING STRATEGIES

Caching stores frequently accessed data closer to users to reduce latency and improve responsiveness.

- Latency reduction
- Performance boost
- Cache consistency issues



22. DECENTRALIZATION

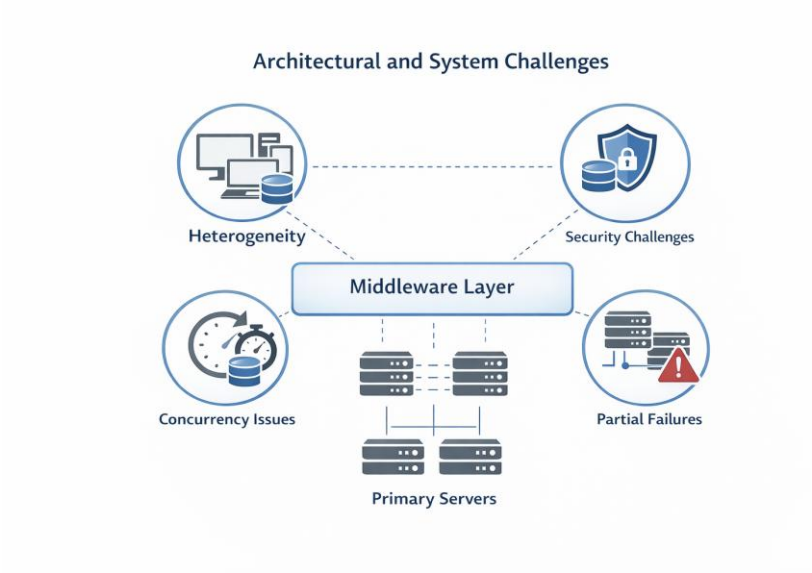
Avoiding central control points reduces bottlenecks and increases system resilience.

- No single point of failure
- Improved robustness
- Autonomous subsystems

ARCHITECTURAL AND SYSTEM CHALLENGES 23. HETEROGENEITY

Distributed systems operate across diverse hardware, operating systems, and programming languages.

- Cross-platform support
- Middleware abstraction
- Protocol compatibility



24. SECURITY CHALLENGES

Security must be enforced across distributed components communicating over untrusted networks.

- Confidentiality
- Integrity
- Authentication
- Authorization

25. CONCURRENCY ISSUES

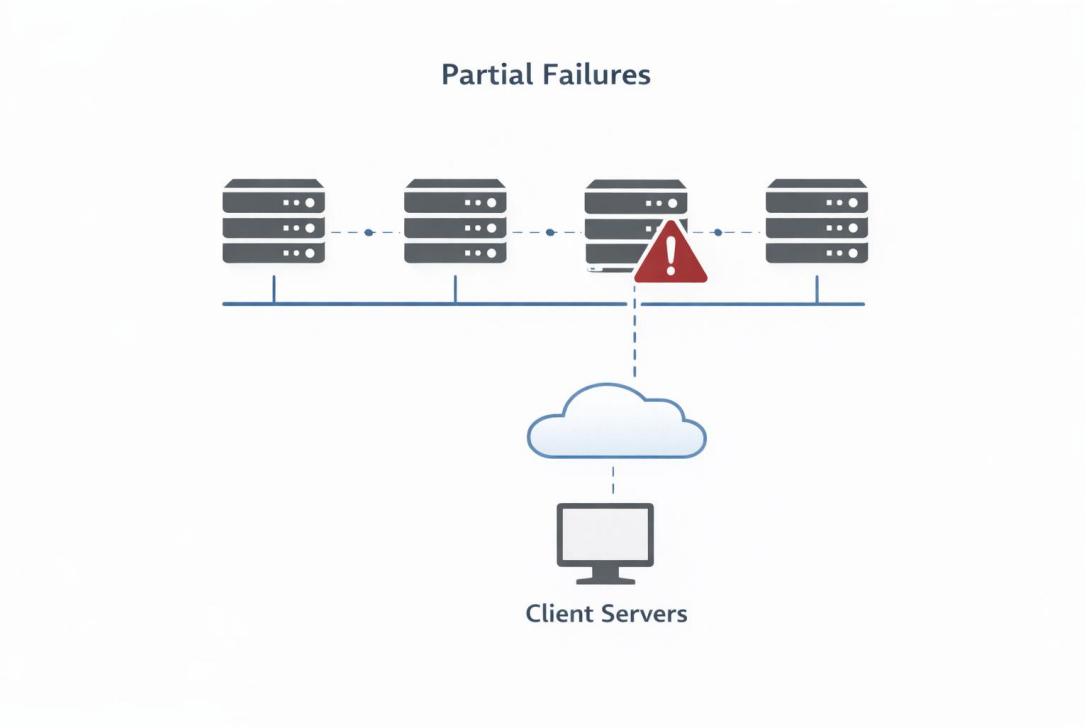
Multiple processes may access shared resources simultaneously, requiring coordination.

- Race conditions
- Deadlocks
- Synchronization mechanisms

26. PARTIAL FAILURES

Components may fail independently while others remain operational, requiring detection and recovery strategies.

- Failure detection
- Redundancy
- Recovery protocols



27. MIDDLEWARE LAYER

Middleware provides communication, naming, security, and transaction services between applications and the OS.

- RPC & messaging
- Naming services
- Security enforcement

TYPES OF DISTRIBUTED SYSTEMS
28. DISTRIBUTED COMPUTING SYSTEMS

These systems focus on computational performance through parallel processing and clusters.

- Clusters
- Grids
- Cloud compute

29. DISTRIBUTED INFORMATION SYSTEMS

Focused on data management and transactions across distributed databases and enterprise platforms.

- Banking systems
- ERP systems
- Transaction processing

30. DISTRIBUTED PERVASIVE SYSTEMS

Operate in dynamic mobile environments such as IoT and smart healthcare systems.

- Mobility
- Wireless networks
- Context awareness

INDUSTRIAL CASE STUDIES
31. CASE STUDY – GOOGLE SEARCH

Google distributes indexed data across global data centers to serve billions of queries daily with minimal latency.

- Data sharding
- Massive replication
- Load balancing

32. GOOGLE – DESIGN INSIGHTS

Google embraces redundancy and decentralization to tolerate failures and maintain continuous service.

- Failure masking
- Geographic redundancy
- Performance optimization

33. CASE STUDY – AMAZON

Amazon's e-commerce platform handles global transactions through microservices and distributed databases.

- High concurrency
- Elastic scaling
- Service isolation

34. AMAZON – CONSISTENCY TRADE-OFFS

Amazon uses eventual consistency in certain services to improve availability and responsiveness.

- CAP trade-offs
- Eventual consistency
- Latency optimization

35. CASE STUDY – WHATSAPP

WhatsApp manages billions of daily messages using asynchronous communication and distributed servers.

- Message queues
- Retry mechanisms
- Global clusters

36. WHATSAPP – RELIABILITY MODEL

Reliability is achieved through message acknowledgments and redundant server clusters.

- Failure transparency
- Mobile heterogeneity
- Scalable messaging

37. CASE STUDY – AWS CLOUD

AWS offers globally distributed infrastructure with availability zones and regional isolation.

- Virtualized resources
- Multi-region replication
- Fault isolation

38. AWS – SCALABILITY ARCHITECTURE

AWS scales horizontally by adding new instances and distributing workloads across regions.

- Elastic compute
- Auto-scaling
- Service abstraction

39. HEALTHCARE IOT SYSTEMS

Healthcare IoT integrates wearable devices with cloud platforms for remote patient monitoring.

- Edge-cloud integration
- Intermittent connectivity
- Security sensitivity

40. SUMMARY

Distributed systems combine transparency, scalability, fault tolerance, and openness to power modern digital ecosystems. Understanding their foundational principles enables deeper exploration of consistency, coordination, and advanced architectures.

- Replication is essential
- Failures are expected
- Trade-offs are unavoidable
- Architecture determines scalability